

Trabalho Academico

Praticas 1 e 2 - Desenvolvimento Full Stack e Versionamento Git

Aluno	Riquelme
Tema	Aplicacao web com HTML, JavaScript e Node.js
Entrega	Codigo no GitHub e relatorio em PDF
Repositorio local	C:\Users\RIQUELME\Documents\HN Joias\faculdade-fullstack-pratica

2026

Resumo da atividade

Este trabalho apresenta uma aplicacao Full Stack simples que demonstra a comunicacao entre cliente e servidor. O usuario digita um nome no front-end, o navegador envia uma requisicao HTTP para o back-end em Node.js e o servidor retorna uma saudacao personalizada em formato JSON.

A segunda parte da pratica foi aplicada com Git, criando commits, uma branch de feature, uma branch de hotfix e merges para integrar as alteracoes na branch principal.

Tecnologias utilizadas

- HTML
- CSS
- JavaScript
- Node.js
- Git
- GitHub

Como executar o projeto

1. Acessar a pasta do projeto.
2. Executar o comando `npm start`.
3. Abrir `http://localhost:3000` no navegador.
4. Digitar um nome e enviar o formulario.

Fluxo de versionamento

O historico Git foi organizado para representar um fluxo comum de desenvolvimento: a branch main recebeu a estrutura inicial, a branch de feature adicionou a interface e a branch de hotfix corrigiu uma validacao do servidor.

Historico Git

```
* 16f7555 (HEAD -> main) merge: integra hotfix de validacao
|\
| * f9fe8f1 (hotfix/validacao-nome) fix: valida tamanho do nome
|/
* 5298c4e merge: integra interface de saudacao
|\
| * 56068d4 (feature/interface-saudacao) feat: adiciona interface de saudacao
|/
* 635081b chore: cria estrutura inicial do servidor
```

Anexos - Código-fonte

Os arquivos abaixo compoem a aplicacao entregue no repositório.

package.json

```
{
  "name": "faculdade-fullstack-pratica",
  "version": "1.0.0",
  "description": "Aplicacao Full Stack simples para demonstrar comunicacao entre front-end e back-end.",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "keywords": [
    "fullstack",
    "nodejs",
    "javascript",
    "html"
  ],
  "author": "Riquelme",
  "license": "MIT"
}
```

server.js

```
const http = require("http");
const fs = require("fs");
const path = require("path");
const { URL } = require("url");

const PORT = process.env.PORT || 3000;
const publicDir = path.join(__dirname, "public");

const contentTypeTypes = {
  ".html": "text/html; charset=utf-8",
  ".css": "text/css; charset=utf-8",
  ".js": "text/javascript; charset=utf-8",
  ".json": "application/json; charset=utf-8"
};

function sendJson(response, statusCode, data) {
  response.writeHead(statusCode, { "Content-Type": contentTypeTypes[".json"] });
  response.end(JSON.stringify(data));
}

function serveStaticFile(response, filePath) {
  const extension = path.extname(filePath);
  const contentType = contentTypeTypes[extension] || "application/octet-stream";

  fs.readFile(filePath, (error, content) => {
    if (error) {
      sendJson(response, 404, { erro: "Arquivo nao encontrado." });
      return;
    }

    response.writeHead(200, { "Content-Type": contentType });
    response.end(content);
  });
}

function handleGreeting(request, response, url) {
  const name = (url.searchParams.get("nome") || "").trim();

  if (!name) {
    sendJson(response, 400, {
      erro: "Informe um nome para receber a saudacao."
    });
  }
}
```

```

    return;
  }

  if (name.length > 60) {
    sendJson(response, 400, {
      erro: "O nome deve ter no maximo 60 caracteres."
    });
    return;
  }

  sendJson(response, 200, {
    mensagem: `Ola, ${name}! Seja bem-vindo(a) a aplicacao Full Stack.`
  });
}

const server = http.createServer((request, response) => {
  const url = new URL(request.url, `http://${request.headers.host}`);

  if (request.method === "GET" && url.pathname === "/api/saudacao") {
    handleGreeting(request, response, url);
    return;
  }

  if (request.method === "GET") {
    const requestedPath = url.pathname === "/" ? "/index.html" : url.pathname;
    const safePath = path.normalize(requestedPath).replace(/^(\\.\\.[/\\])+/, "");
    const filePath = path.join(publicDir, safePath);

    serveStaticFile(response, filePath);
    return;
  }

  sendJson(response, 405, { erro: "Metodo nao permitido." });
});

server.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});

```

public/index.html

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pratica Full Stack</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <section class="panel">
      <p class="eyebrow">Desenvolvimento Full Stack</p>
      <h1>Saudacao personalizada</h1>
      <p class="intro">
        Digite seu nome para enviar uma requisicao HTTP ao servidor Node.js e receber uma resposta
        personalizada.
      </p>

      <form id="greeting-form" class="form">
        <label for="name">Nome</label>
        <div class="input-row">
          <input id="name" name="name" type="text" placeholder="Ex.: Ana" autocomplete="name" required>
          <button type="submit">Enviar</button>
        </div>
      </form>

      <div id="result" class="result" aria-live="polite">
        A resposta do servidor aparecera aqui.
      </div>
    </section>
  </main>

```

```
<script src="script.js"></script>
</body>
</html>
```

public/style.css

```
:root {
  color-scheme: light;
  font-family: Arial, Helvetica, sans-serif;
  background: #eef2f7;
  color: #172033;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
}

.app {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 32px 16px;
}

.panel {
  width: min(100%, 680px);
  background: #ffffff;
  border: 1px solid #d7deea;
  border-radius: 8px;
  padding: 32px;
  box-shadow: 0 18px 45px rgba(23, 32, 51, 0.12);
}

.eyebrow {
  margin: 0 0 10px;
  color: #2f6fed;
  font-size: 0.78rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0;
}

h1 {
  margin: 0 0 12px;
  font-size: 2rem;
  line-height: 1.15;
}

.intro {
  margin: 0 0 24px;
  color: #4d5b73;
  line-height: 1.6;
}

.form {
  display: grid;
  gap: 10px;
}

label {
  font-weight: 700;
}

.input-row {
  display: grid;
  grid-template-columns: 1fr auto;
```

```

    gap: 10px;
  }

  input,
  button {
    min-height: 44px;
    border-radius: 6px;
    font: inherit;
  }

  input {
    width: 100%;
    border: 1px solid #b9c3d4;
    padding: 0 14px;
  }

  input:focus {
    border-color: #2f6fed;
    outline: 3px solid rgba(47, 111, 237, 0.16);
  }

  button {
    border: 0;
    padding: 0 18px;
    background: #2f6fed;
    color: #ffffff;
    font-weight: 700;
    cursor: pointer;
  }

  button:hover {
    background: #2459c8;
  }

  .result {
    margin-top: 22px;
    min-height: 48px;
    padding: 14px 16px;
    border-left: 4px solid #2f6fed;
    background: #f5f8ff;
    color: #24324a;
    line-height: 1.5;
  }

  .result.error {
    border-left-color: #c43d3d;
    background: #fff5f5;
    color: #8b2424;
  }

  @media (max-width: 560px) {
    .panel {
      padding: 24px;
    }

    .input-row {
      grid-template-columns: 1fr;
    }

    button {
      width: 100%;
    }
  }

```

public/script.js

```

const form = document.querySelector("#greeting-form");
const nameInput = document.querySelector("#name");
const result = document.querySelector("#result");

form.addEventListener("submit", async (event) => {
  event.preventDefault();

```

```

const name = nameInput.value.trim();
result.classList.remove("error");
result.textContent = "Enviando requisicao ao servidor...";

try {
  const response = await fetch(`/api/saudacao?nome=${encodeURIComponent(name)}`);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.erro || "Nao foi possivel gerar a saudacao.");
  }

  result.textContent = data.mensagem;
} catch (error) {
  result.classList.add("error");
  result.textContent = error.message;
}
});

```

README.md

Pratica Full Stack e Versionamento

Projeto academico para demonstrar a comunicacao entre front-end e back-end usando HTML, CSS, JavaScript e Node.js.

Funcionalidades

- Pagina HTML com formulario para digitar o nome do usuario.
- Requisicao HTTP feita pelo JavaScript do navegador.
- Rota `GET /api/saudacao` no servidor Node.js.
- Resposta JSON com saudacao personalizada.
- Historico Git com uso de branch de feature e branch de hotfix.

Como executar

1. Instale o Node.js.
2. No terminal, acesse a pasta do projeto.
3. Execute:

```

```bash
npm start
```

```

4. Abra no navegador:

```

```text
http://localhost:3000
```

```

Rotas

| Metodo | Rota | Descricao |
|--------|----------------------------|---|
| GET | / | Exibe a interface da aplicacao. |
| GET | /api/saudacao?nome=SeuNome | Retorna uma saudacao personalizada em JSON. |

Fluxo de versionamento utilizado

- `main`: branch principal do projeto.
- `feature/interface-saudacao`: branch usada para criar a interface e integrar com o servidor.
- `hotfix/validacao-nome`: branch usada para corrigir a validacao de nome vazio.

Autor

Riquelme